

BACKEND FOR HIGHLOADED ENVIRONMENT · WEEK 2

PRACTICE · 1 HOUR · 1 POINT

Load Testing HTTP/1.1 vs HTTP/2 vs HTTP/3

Connection pooling and the cost of the TLS handshake

Java 21 · Spring Boot

k6 · curl · h2load

Nurov Arnur

ONE HOUR

Turn Today's Lecture Into Numbers

Three claims from the lecture. By the end of the week each one should be a measurement from your own machine, with a mechanism attached to it.

CLAIM 1

A cold connection costs round trips

Measure the same endpoint with connection reuse on and off. The gap in p50 is the handshake.

CLAIM 2

HTTP/2 wins on concurrency, not on one request

One request at a time should show almost no difference. Fifty in parallel should.

CLAIM 3

Loss is what separates HTTP/2 from HTTP/3

On a clean link they are close. Add 100 ms delay and 1 % loss and the ranking should change.

Absolute numbers are worthless here — your laptop is not a server. Differences between configurations are the whole exercise.

SETUP

The Rig You Are Building

Load generator

k6 · h2load · curl

Fixed arrival rate, warm-up excluded



Impairment

tc netem

delay 100 ms, loss 1 %



Edge

Caddy or nginx

TLS 1.3 · h3, h2, http/1.1



Service

Spring Boot, Java 21

Week 1 baseline, one JSON endpoint

Why an edge at all

Spring Boot serves HTTP/1.1 and HTTP/2 directly, but not HTTP/3. Caddy speaks all three and needs three lines of config, so it is the cheapest way to get a comparable HTTP/3 endpoint.

Keep it honest

Same endpoint, same payload, same concurrency, same duration for every run. Change exactly one variable at a time or the numbers mean nothing.

PRACTICE 2 · PART ONE OF FOUR

01

Serving All Three Versions

A certificate

Spring Boot h2

Caddy h3

Smoke test

STEP 1

A Certificate, Because h2 Means HTTPS

KEYTOOL — SELF-SIGNED, LAB ONLY

```
keytool -genkeypair \  
  -alias practice -keyalg EC -groupname secp256r1 \  
  -storetype PKCS12 -keystore keystore.p12 \  
  -storepass changeit -validity 365 \  
  -dname "CN=localhost" \  
  -ext "SAN=dns:localhost,ip:127.0.0.1"
```

Put it in [src/main/resources](#). An EC key is deliberate: the handshake signature is cheaper than RSA, which matters once you are opening thousands of connections per second.

The SAN matters

Without a subject alternative name, modern clients reject the certificate outright and you will spend the hour debugging TLS instead of measuring it.

Trust, do not disable

In the lab you may skip verification (`curl -k, k6 insecureSkipTLSVerify`). Never in a report that claims to be about production behaviour, and never in code you keep.

Why no plaintext run

HTTP/2 over cleartext exists in the specification and essentially nowhere else; HTTP/3 encrypts by definition. So TLS is part of every comparison you make today.

STEP 2

Spring Boot — One Port, Two Versions

APPLICATION.YML

```
server:
  port: 8443
  http2:
    enabled: true
  ssl:
    enabled: true
    key-store: classpath:keystore.p12
    key-store-type: PKCS12
    key-store-password: changeit
tomcat:
  threads:
    max: 200
  max-connections: 8192
  accept-count: 100
```

ALPN does the choosing

The server advertises h2, http/1.1 inside the TLS handshake. One port, and the client decides which version it gets — which is why the protocol is a variable in your load test, not a deployment.

Write down the three limits

threads.max caps concurrent blocking requests, max-connections caps open sockets, accept-count is the backlog from part one of the lecture. Your report must state them, because they are your ceiling.

Watch for the silent downgrade

Miss the TLS block and HTTP/2 will not negotiate. You get a perfectly working service, a perfectly plausible graph, and both runs are HTTP/1.1.

STEP 3

The Endpoint Under Test

QUOTECONTROLLER.JAVA

```
record Quote(long id, String pair, double bid) {}

@RestController
@RequestMapping("/api")
class QuoteController {

    @GetMapping("/quote/{id}")
    Quote one(@PathVariable long id) {
        return new Quote(id, "USDKZT", 533.14);
    }

    // n varies the payload: 1 row ~90 B, 200 rows ~18 KB
    @GetMapping("/quotes")
    List<Quote> many(@RequestParam(defaultValue="1") int n) {
        return LongStream.rangeClosed(1, n)
            .mapToObj(i -> new Quote(i, "USDKZT", 533.14))
            .toList();
    }
}
```

No work in the handler

No database, no sleep, no logging per request. You are measuring the protocol; anything the handler does is noise on top of the signal you want.

Two payload sizes, on purpose

With a 90-byte body, headers are a large share of the bytes on the wire — so HPACK shows up. With 18 KB, the initial congestion window from the lecture starts to matter.

Keep the body byte-identical

No timestamps, no random values. A body whose length changes between runs changes the bytes on the wire, and you will read that as protocol behaviour.

STEP 4

An Edge That Speaks HTTP/3

CADDYFILE

```
{
  auto_https disable_redirects
}

localhost:9443 {
  tls cert.pem key.pem
  reverse_proxy https://localhost:8443 {
    transport http {
      tls_insecure_skip_verify
      versions h2c 2
    }
  }
}
```

HTTP/3 is on by default on a TLS listener, so port 9443 answers h3, h2 and http/1.1 at once.

Open UDP 9443

QUIC is UDP. A firewall rule that only allows TCP gives you a silent fallback to HTTP/2 and a comparison that measures nothing.

Raise the UDP buffers on Linux

If the log warns about the receive buffer, raise `net.core.rmem_max`. Otherwise the kernel drops datagrams under load and you will blame QUIC for it.

Record the extra hop

Through the proxy every request crosses one more process. That is fine — as long as all three protocol runs go through the same edge, and you say so in the report.

STEP 5 · DO NOT SKIP THIS

Prove Which Version You Are Speaking

```
# force each version and print only the negotiated one
curl -k --http1.1 -so /dev/null -w '%{http_version}\n' https://localhost:9443/api/quote/1
curl -k --http2 -so /dev/null -w '%{http_version}\n' https://localhost:9443/api/quote/1
curl -k --http3 -so /dev/null -w '%{http_version}\n' https://localhost:9443/api/quote/1

# expected: 1.1 / 2 / 3 - anything else means your rig is lying to you
```

Got 1.1 when asking for 2?

ALPN did not offer h2 — usually TLS is off, or you are on the plaintext port.

curl: option --http3 unknown?

Your curl was not built with HTTP/3. Check `curl -V` for HTTP3, or run curl from a container that has it.

Also worth printing

`%{time_connect},%
{time_appconnect},%
{time_starttransfer}` — TCP, TLS
and first byte, separated.

PRACTICE 2 · PART TWO OF FOUR

02

The Client Side, in Java

HttpClient Pooling Timeouts A percentile harness

Asking for a Version, and Checking What You Got

ONE CLIENT FOR THE WHOLE PROCESS

```
static final HttpClient CLIENT = HttpClient.newBuilder()
    .version(Version.HTTP_2)      // preference, not promise
    .connectTimeout(Duration.ofSeconds(2))
    .sslContext(labTrustAll())    // self-signed cert, lab
only
    .build();

var req = HttpRequest.newBuilder()
    .uri(URI.create("https://localhost:9443/api/quote/1"))
    .timeout(Duration.ofMillis(500))
    .GET().build();

var res = CLIENT.send(req, BodyHandlers.ofString());

// assert, do not assume
if (res.version() != Version.HTTP_2)
    throw new IllegalStateException("got " + res.version());
```

Version is negotiated

Over TLS the JDK offers h2 via ALPN and falls back to 1.1 without telling you. `res.version()` is the only honest answer.

No HTTP/3 in the JDK

Java 21's built-in client tops out at HTTP/2. For h3 you need an external client — that is why the HTTP/3 numbers come from curl or h2load, not from your Java harness. Say so in the report.

Two timeouts, two meanings

`connectTimeout` bounds the handshake, the request timeout bounds the whole exchange. Neither has a default worth relying on.

THE BUG YOU WILL FIND IN REAL CODE

A Client Per Request Is a Handshake Per Request

WRONG — NEW POOL EVERY TIME

```
for (int i = 0; i < n; i++) {  
    // fresh client → fresh pool → TCP + TLS again  
    var c = HttpClient.newHttpClient();  
    c.send(req, BodyHandlers.discarding());  
}
```

Each iteration pays the round trips from the lecture. Over a 100 ms link that is 200 ms of pure setup per call.

RIGHT — ONE CLIENT, REUSED

```
// created once, shared, never closed per call  
for (int i = 0; i < n; i++) {  
    CLIENT.send(req, BodyHandlers.discarding());  
}
```

The JDK client keeps a connection per destination; with HTTP/2 all concurrent requests share one connection as streams.

If you need explicit pool limits, use Apache HttpClient 5

```
var pool = PoolingHttpClientConnectionManagerBuilder.create()  
    .setMaxConnTotal(100).setMaxConnPerRoute(20)    // per-route is the one that bites  
    .setDefaultConnectionConfig(ConnectionConfig.custom()  
        .setValidateAfterInactivity(TimeValue.ofSeconds(5)).build())  
    .build();
```

The JDK client has no pool-size knob. When a report needs “20 connections, no more”, this is the library that gives it to you.

SIZING, NOT GUESSING

How Many Connections Do You Need

$$L = \lambda \times W$$

Concurrent calls in flight equals arrival rate times time per call. That number — not your CPU count, not a round figure — is the pool size.

500 RPS · 40 MS

20

connections in flight. A pool of 200 buys you nothing but idle sockets.

500 RPS · 400 MS

200

Ten times slower downstream means ten times the pool — or requests start queueing for a connection.

SAME LOAD, HTTP/2

1

connection, 20 concurrent streams on it. Pool sizing becomes stream limits — check `SETTINGS_MAX_CONCURRENT_STREAMS`.

Queueing for a connection is invisible in server metrics: the server sees a request that arrived late, and your client sees latency it cannot explain. Measure the wait for a pool slot separately.

WEEK 1, REUSED

A Percentile Harness, Twenty Lines

```
int warmup = 2_000, n = 20_000;
for (int i = 0; i < warmup; i++)
    CLIENT.send(req, BodyHandlers.discarding()); // JIT +
pool warm

long[] us = new long[n];
for (int i = 0; i < n; i++) {
    long t0 = System.nanoTime();
    CLIENT.send(req, BodyHandlers.discarding());
    us[i] = (System.nanoTime() - t0) / 1_000;
}

Arrays.sort(us);
System.out.printf("p50=%d p95=%d p99=%d max=%d us%n",
    us[(int) (n * 0.50)], us[(int) (n * 0.95)],
    us[(int) (n * 0.99)], us[n - 1]);
```

Sorting the full array is fine at this scale and has no estimation error. At millions of samples you would use HdrHistogram instead.

Know what this does not measure

One request at a time means concurrency of one. It cannot show multiplexing, queueing or saturation — and HTTP/2 will look identical to HTTP/1.1 here. That is the correct result, not a bug.

Warm up, always

The first thousand calls measure the JIT, class loading and an empty pool. Reporting them is reporting JVM start-up as protocol latency.

Use it for the cold-versus-warm run

Swap the shared client for a per-request one and rerun. The difference in p50 is your handshake number, isolated from any load effect.

PRACTICE 2 · PART THREE OF FOUR

03

Measuring Properly

k6 Arrival rate Reading the output h2load netem

THE LOAD SCRIPT

k6 — Fixed Arrival Rate, Not Fixed Users

LOAD.JS

```
import http from 'k6/http';

export const options = {
  insecureSkipTLSVerify: true,
  noConnectionReuse: __ENV.COLD === '1', // the cold run
  summaryTrendStats: ['p(50)', 'p(95)', 'p(99)', 'max'],
  scenarios: {
    steady: {
      executor: 'constant-arrival-rate',
      rate: 500, timeUnit: '1s', duration: '60s',
      preAllocatedVUs: 200, maxVUs: 800,
    },
  },
};

export default function () {
  http.get(`${__ENV.URL}/api/quote/1`);
}
```

Two runs from one script

COLD=1 turns off connection reuse. Same script, same load, one variable changed — that is the handshake experiment.

k6 covers 1.1 and 2

It negotiates HTTP/2 over TLS automatically; force 1.1 by pointing at an endpoint that only offers `http/1.1` in ALPN. Verify with the `http_req_duration` tag for protocol.

HTTP/3 needs another tool

Stock k6 does not generate h3. Use `h2load` built with QUIC support, or a scripted curl loop. If you cannot get either working, report the single-request curl comparison and state the limitation.

READING THE OUTPUT

The Metrics That Map to the Lecture

K6 METRIC	WHAT IT IS	WHAT IT TELLS YOU TODAY
<code>http_req_blocked</code>	Waiting for a free connection	Non-zero means the pool is the bottleneck, not the server.
<code>http_req_connecting</code>	TCP handshake	Should be ~0 on the warm run and one RTT on the cold one.
<code>http_req_tls_handshaking</code>	TLS handshake	This is the number the task asks you to name.
<code>http_req_waiting</code>	Time to first byte	The server's own contribution. Should barely move between protocols.
<code>http_req_duration</code>	The whole request	Report p50, p95, p99 — never the mean.
<code>dropped_iterations</code>	Load k6 could not send	If this is non-zero, your generator ran out of VUs — the run is invalid.

Warm minus cold on `connecting` + `tls_handshaking` is the cost of a cold connection, measured rather than quoted from a slide.

THE TOOLBOX

Which Tool Speaks Which Version

k6

1.1 · 2

Scriptable, honest arrival-rate executors, percentile output built in.
Your primary tool for this task.

wrk

1.1 only

Very high throughput per core. `wrk -t4 -c50 -d60s --latency URL`. Cannot do HTTP/2, so it cannot be your whole method.

h2load (nghttp2)

1.1 · 2 · 3*

`h2load -n 50000 -c 50 -m 20 URL`. HTTP/3 only in a build with QUIC support — check `h2load --version`.

curl

1.1 · 2 · 3

Not a load generator, but the only tool that reliably proves which version you got and splits the phase timings per request.

Name your tools in the report

Version, flags, and which machine ran them. A measurement whose tooling is not stated cannot be reproduced, and an unreproducible measurement is an opinion.

MAKE LOCALHOST BEHAVE LIKE THE INTERNET

Adding Delay and Loss

TC NETEM — INSIDE THE CONTAINER OR VM

```
# needs NET_ADMIN: docker run --cap-add=NET_ADMIN
```

```
...
```

```
# 100 ms each way, 1 % loss, a little jitter
```

```
tc qdisc add dev eth0 root netem \  
    delay 100ms 10ms loss 1%
```

```
# check it took effect
```

```
ping -c 3 localhost    # expect ~200 ms RTT
```

```
# always clean up before the next run
```

```
tc qdisc del dev eth0 root
```

On macOS or Windows run the server in a Linux container; netem is a Linux qdisc and has no direct equivalent on the host.

Delay alone changes the handshake story

At 200 ms RTT, the cold-versus-warm gap stops being a curiosity and becomes the dominant term. That is the lecture's first slide, reproduced on your laptop.

Loss is what separates h2 from h3

With many concurrent streams on one TCP connection, a dropped packet stalls all of them. Watch p99 in particular — this effect lives in the tail, not in the median.

One variable at a time

Run delay-only and delay-plus-loss as separate rows. Changing two things at once gives you a number you cannot attribute to anything.

PRACTICE 2 · PART FOUR OF FOUR

04

The Task

Four runs One table Three explanations Due end of week

What You Are Handing In

- | | | |
|-----------|---|---|
| 01 | Three protocol runs | Same endpoint, same arrival rate, same duration, over HTTP/1.1, HTTP/2 and HTTP/3. Report p50, p95, p99 and achieved RPS for each, plus the negotiated version proved by curl. |
| 02 | The cost of the handshake | Repeat one protocol with connection reuse on and off. Name the handshake cost in milliseconds and show which metric you took it from. |
| 03 | The same, on a bad network | Add 100 ms delay and 1 % loss with netem and repeat the three protocol runs. State which version degrades least and by how much. |
| 04 | Three explanations — the actual work | Two or three sentences per result, each naming a mechanism from the lecture: handshake round trips, initial congestion window, multiplexing, head-of-line blocking, HPACK. "Newer is faster" earns nothing. |

Deliverable: a repository with the service, the Caddyfile, the k6 script and the netem commands, plus `REPORT.md`. If you could not get HTTP/3 load generation working, say so explicitly and report the curl comparison instead — an honest limitation costs nothing.

The Table to Fill In

PROTOCOL	CONNECTION	P50	P95	P99	RPS	MECHANISM THAT EXPLAINS IT
HTTP/1.1	reused	—	—	—	—	baseline
HTTP/1.1	new per request	—	—	—	—	TCP + TLS round trips = ... ms
HTTP/2	reused	—	—	—	—	multiplexing, HPACK
HTTP/3	reused	—	—	—	—	QUIC handshake, per-stream loss recovery
each of the three	netem 100 ms / 1 %	—	—	—	—	head-of-line blocking — expect it in p99

Above the table state: hardware, JDK version, tool versions, arrival rate, duration, payload size, and the Tomcat limits from your yml. Below it, the three explanations.

HOW THE POINT IS AWARDED

What Earns It, What Loses It

EARNs THE POINT

All four runs present, with the environment and the load parameters stated.

Percentiles, not averages, and the negotiated protocol proved rather than assumed.

A handshake cost in milliseconds, taken from your own two runs.

Explanations that name a mechanism — including an honest “this result is not what the lecture predicted, and here is why I think that is”.

A reproducible command line and a README stating where AI was used.

LOSES IT

Three runs that were all actually HTTP/1.1, because ALPN never offered anything else.

A ten-second run with no warm-up, or a run with non-zero `dropped_iterations`.

A load generator pinned at 100 % CPU — you measured your laptop, not the protocol.

Averages only, or a chart with no axis units.

A result you cannot explain when asked. That is zero regardless of how the report looks.

BEFORE YOU LEAVE

Checklist and What This Feeds

WORKING BEFORE YOU GO HOME

Three curl commands returning 1.1, 2 and 3.

k6 running one 60-second scenario without dropped iterations.

netem adding and removing cleanly on the server container.

A Java client that asserts `res.version()` instead of hoping.

WHERE THIS COMES BACK

Next week: gRPC on the HTTP/2 streams you just measured, plus Protobuf, idempotency, timeouts and retries.

SIS #1, week 6: the protocol choice in your design document has to be justified by numbers like these.

SIS #2, week 8: the pooling and timeout code from today is the client you will submit measurements for.

Submissions and questions in MS Teams. Late work receives 50 % of the task value.